

NivXRay · Single-IUE Convergence — Design-Only Blueprint

Type: DESIGN ONLY. No code, adapter, IUE, router, MITRE, verdict, canonical-evidence, or Workspace change is proposed or performed. **Date:** 2026-02-15 · Session-20 (post ADR-0013 audit ratification). **Anchor:** ADR-0013 · IUE 360° READ-ONLY Audit — CONFIRMED. **Owner constraint:** IUE decides **WHAT and WHY**. Router decides **HOW TO EXECUTE**. Adapters acquire/normalize. Analyzers analyze. Canonical Evidence preserves provenance. Correlation correlates. MITRE/Verdict remain authoritative. Workspace projects the canonical result. **Nothing else.**

LOCKS in force: M1–M8 implementation is LOCKED. Every other lock from ADR-0013 remains intact.

§1 · CURRENT IUE contract (frozen for the record)

The observable IUE decision object is `InputUnderstanding` at `services/die/input_understanding.py:122`.
Reproduced from live code:

```
InputUnderstanding (dataclass · frozen for the record 2026-02-15) ■■■ input_type : str #
21-value flat enum ■■■ hero : str # display label ■■■ confidence : float # 0.0-1.0 ■■■
reasoning : List[str] # prose bullets, no structure ■■■ summary : ContentSummary ■ ■■■ length :
int ■ ■■■ entropy : float ■ ■■■ ascii_ratio : float ■ ■■■ (no compound-content dimensions)
■■■ decode_required : bool ■■■ decode_next : str # prose label ■■■ decode_layers :
List[DecodeLayerPlan] ■ ■■■ index : int ■ ■■■ name : str ■ ■■■ output_preview : str ■ ■■■
confidence : float ■■■ plan : List[PlanStep] ■ ■■■ id : str ■ ■■■ description : str ■ ■■■
engine : str # prose label (not an id) ■ ■■■ status : str ■ ■■■ reason : str ■■■
execution_trace : List[str] # ordered log strings ■■■ overall_status : str ■■■ engines_selected
: List[str] # prose labels ■■■ engines_skipped : List[str] # prose labels ■■■ pipeline_flow :
List[str] # prose labels
```

Contract observations locked here:

- Everything downstream of `input_type` is either a prose label or a scalar. There is no registry-referenceable adapter id or analyzer id.
- Source / vendor / content-composition / artifact-expected-set / acquisition-intent / OCR-intent / merge-strategy are **absent**.
- Confidence is scalar and lives only on `input_type` and each `decode_layer`. There is no per-decision confidence chain.
- Reasoning is a prose list. No structured provenance.
- Plan steps carry only a prose "engine" label — the router that consumes them has no machine-readable identifier to look up.

Anything the target contract adds must be **additive** to this frozen shape so existing consumers (Workspace UI panels, `investigation_results.py`, `canonical.py`, `nivxforge` mirror) do not break during migration.

§2 · TARGET IUE decision contract — hierarchical model (design)

Owner's explicit correction: **do not flatten every dimension onto the top level**. Structure as a small, composable hierarchy.

2.1 Top-level shape

```
InputUnderstanding ■■■ envelope // metadata about the classification itself ■■■ input // WHAT
is the input ■■■ source // WHERE did it come from ■■■ content // WHAT DOES IT CONTAIN
(compound) ■■■ intent // WHY / what should happen next ■■■ plan // HOW the router should
execute ■■■ provenance // WHO/WHAT decided this – audit chain
```

2.2 envelope — meta about the classification

Fields: `iue_version` (string, e.g. `iue.v3.0`), `deterministic`: `bool` (this IUE never calls LLM or network — must stay true), `classifier_chain`: `List[str]` (which sub-classifiers voted), `classified_at`: `ISO8601`, `evidence_ref` (deterministic hash of input for de-duplication).

Purpose: lets the Workspace show *how* the IUE reached its verdict and enables cache-keying on identical inputs.

2.3 input — WHAT it is (single-dimension answer)

```
input ■■■ type : enum(url | file | text | log | archive | binary | mixed | unknown) ■■■ format
: enum(html | pdf | docx | pptx | xlsx | pe | elf | evtx | ■ sysmon_xml | vendor_json | ndjson |
cef | leef | ■ stix | csv | plain_text | powershell | cmd | python | ■ bash | javascript |
vbscript | base64_blob | hex_blob | ■ gzip_blob | registry_export | image | pcap | eml | msg |
...) ■■■ format_source : enum(magic_bytes | filename | mime | heuristic) ■■■ confidence : float
■■■ size : int
```

Contract rule: `type` is the coarse category. `format` is the fine-grained format detected. Both are always present. No prose labels — enums only, so the router can dispatch machine-fast.

2.4 source — WHERE / WHO produced it (independent dimension)

```
source ■■■ origin : enum(uploaded | pasted | urlref | recursion | telemetry_stream) ■■■ vendor
: enum(sysmon | crowdstrike | defender | sentinel_one | ■ talos | mandiant | bleepingcomputer |
medium | ■ systemweakness | infosecwriteups | github | gitlab | ■ ... | unknown_vendor |
not_applicable) ■■■ vendor_source : enum(host_catalogue | telemetry_header | prose_marker | ■
filename_pattern | not_applicable) ■■■ locator : str // URL / filename / GridFS ref ■■■
parent_ref : Optional[str] // if origin=recursion, the parent evidence_ref ■■■ confidence :
float
```

Design point: `vendor_catalogue` becomes an *input* to a first-class dimension rather than a hidden gate. This makes it visible in Workspace and auditable.

2.5 content — WHAT IT CONTAINS (compound, list-of-facets)

Compound and list-shaped because a single input can contain many content facets simultaneously — the case that broke SystemWeakness and PrevMode.

```
content ■■■ has_text : bool ■■■ has_encoded : bool ■■■ has_images : bool ■■■ has_artifacts :
bool // linked / embedded artifacts ■■■ has_commands : bool ■■■ has_ioc_atoms : bool ■■■
has_binary : bool ■■■ facets : List[ContentFacet] ■■■ confidence : float ContentFacet ■■■ kind
: enum(text | encoded | image_ref | artifact_ref | ■ command_line | ioc_atom | binary_region |
archive_member) ■■■ extraction_hint : str // e.g. "base64 in article body", "img src in <p>"
■■■ expected_producer: enum(article_text | image_ocr | archive_member | ■ telemetry_field |
decoder_layer | manual_paste) ■■■ observed_ref : Optional[str] // pointer into the actual bytes
■■■ confidence : float
```

Design point: because content is a *list* of facets, one URL can carry [`text`, `encoded_command`, `image_ref`, `artifact_ref`] all as siblings. The plan can then fan out one execution step per facet, all rejoining at canonical evidence.

2.6 intent — WHY (what actions this input requires)

```
intent ■■■ acquire : bool // fetch from network ■■■ decompose : bool // break into content facets ■■■ decode : bool // b64/hex/gzip/xor peel ■■■ ocr : bool // image → text ■■■ analyze : bool // language/AST analysis ■■■ correlate : bool // cross-facet correlation ■■■ inspect_only : bool // atomic IOC – enrich, do not analyze ■■■ confidence : float
```

Design point: intents are booleans, not a single enum, because they compose. A SystemWeakness URL is {acquire=true, decompose=true, decode=true, ocr=true, analyze=true, correlate=true, inspect_only=false} — all at once.

2.7 plan — HOW the router should execute

```
plan ■■■ steps : List[PlanStep] ■■■ final_convergence: str // canonical evidence sink id (fixed) PlanStep ■■■ step_id : str // deterministic, stable across reruns ■■■ adapter_id : Optional[str] // registry id – NOT prose ■■■ analyzer_id : Optional[str] // registry id – NOT prose ■■■ input_ref : str // parent step_id or "root" ■■■ content_facet : Optional[int] // index into content.facets[] ■■■ depends_on : List[str] // step_ids ■■■ failure_policy : enum(fail_loud | fail_forward | log_and_continue) ■■■ required : bool ■■■ reason : str // prose (human-facing only)
```

Design point: every step references a machine-readable id in an adapter/analyzer registry (§5). The prose `reason` is analyst-facing only; the router NEVER dispatches on prose.

2.8 provenance — the audit chain of the decision

```
provenance ■■■ decisions : List[Decision] ■■■ conflicts : List[Conflict] // where sub-classifiers disagreed Decision ■■■ field_path : str // "input.format", "source.vendor", "intent.ocr", ... ■■■ decided_by : enum(magic_bytes_detector | vendor_catalogue | ■ content_probe | intent_policy | user_override) ■■■ evidence : str // machine-parsable justification ■■■ confidence : float ■■■ decided_at : ISO8601 Conflict ■■■ field_path : str ■■■ voters : List[str] ■■■ choices : List[str] ■■■ resolution : str // "highest_confidence" | "policy_default" | ... ■■■ notes : str
```

Design point: every field the IUE emits carries its "who decided this and why". This is the record the analyst can drill into when they ask "why did NivXRay think this URL was atomic?".

2.9 Contract discipline (must-hold invariants)

1. **Deterministic.** IUE must remain LLM-free and network-free. `envelope.deterministic=true` is a contract, not a hint.
2. **Additive.** Every existing consumer of the old `InputUnderstanding` fields must still find them (either verbatim or via a stable projection). See §14.
3. **No downstream implementation details.** No MITRE technique ids, no verdict scores, no adapter internal state, no cache keys of downstream engines. IUE decides WHAT/WHY. Downstream layers decide HOW/RESULT.
4. **Idempotent on identical input.** Same bytes ⇒ same envelope, byte-identical.
5. **Total** on any input. Every unrecognised input still gets `type=unknown`, `format=plain_text`, `source.origin=pasted`, `source.vendor=not_applicable`, `content.facets=[{kind: text}]`, `intent.inspect_only=true`, `plan=[{analyzer_id: die.default, failure_policy: log_and_continue}]`. No exceptions bubble up.

§3 · Current vs Target — comparison table

Concern	CURRENT (services/die/input_understanding.py)	TARGET (design)
Determinism	■	■ (locked as contract, <code>envelope.deterministic=true</code>)
Type dimension	■ (input_type — flat 21-enum, mixes type + format + language)	■ (input.type coarse enum + input.format fine enum, separated)
Source dimension	■ absent	■ source.origin + source.vendor + source.vendor_source + source.locator
Content dimension	■ collapsed into input_type	■ content.facets: List[ContentFacet] — compound, list-of-facets
Intent dimension	■ implicit in _next_engine prose	■ intent.{acquire, decompose, decode, ocr, analyze} — composable booleans
Adapter id	■ prose label	■ plan.steps[].adapter_id — registry id
Analyzer id	■ prose label	■ plan.steps[].analyzer_id — registry id
Provenance	■ prose reasoning[]	■ provenance.decisions[] — structured {field_path, decided_by, evidence, confidence}
Conflict tracking	■ silent single-classifier	■ provenance.conflicts[] — records when sub-classifiers disagreed
Idempotence	■■ effectively yes but not asserted	■ contract-locked; envelope.evidence_ref is deterministic hash
Recursive re-entry	■ recursion is DIE-internal, IUE not consulted	■ source.origin=reursion + source.parent_ref — child artifacts re-enter IUE
Failure semantics	■■ one overall_status scalar	■ per-step failure_policy + required flag
Consumers	Workspace UI + investigation_results.py + canonical.py + nivxforge mirror	same set; the current field surface must remain as a stable projection (§14)

§4 · Universal Input Router — design

Owner constraint: the router is a **thin dispatcher**. It does not classify, does not analyze, does not decide policy. It executes a plan the IUE already built.

4.1 Responsibilities

- Read `InputUnderstanding.plan.steps[]`.

- For each step, look up `adapter_id` / `analyzer_id` in the registries (§5).
- Enforce `depends_on` topological order.
- Handle `failure_policy` (fail-loud / fail-forward / log-and-continue).
- Attach every emitted evidence record to the originating `step_id` (for provenance).
- STOP at `final_convergence` and hand aggregated evidence to canonical evidence + correlation.

4.2 Non-responsibilities

- Does NOT decide "should we acquire this URL?" — that's IUE `intent.acquire`.
- Does NOT decide "should we OCR this image?" — that's IUE `intent.ocr`.
- Does NOT pick between competing analyzers — the IUE plan already chose one.
- Does NOT compute confidence — analyzers do.
- Does NOT emit MITRE — analyzers/correlator/authoritative surface do.

4.3 API sketch (design only)

```
router.execute(iu: InputUnderstanding, raw_input: bytes|str, ctx: ExecutionContext) → RouterResult: evidence_records: List[CanonicalEvidenceRecord] step_results: Dict[step_id, StepResult] router_trace: List[str] terminated: enum(complete | partial | failed)
```

4.4 Router registry layers (single dispatcher, three lookup tables)

```
router ■■■ adapter_registry : Dict[adapter_id, AdapterInterface] ■■■ analyzer_registry : Dict[analyzer_id, AnalyzerInterface] ■■■ convergence_sink : CanonicalEvidenceSinkInterface
```

Everything the router can do lives behind these three tables. Adding a new capability = registering a new adapter or analyzer. Removing one = de-registering. The router itself never changes.

4.5 What replaces `_atomic_ioc_kind`?

Under the target design, `_atomic_ioc_kind` disappears. The IUE emits `intent.inspect_only=true` when the input is a bare atomic IOC (URL/IP/hash/filename). The router then dispatches to `analyzer_id="ioc_enrichment"`. The short-circuit still happens — but it's the IUE's explicit decision, visible in provenance, not a silent guard.

§5 · Adapter / Analyzer registries — design

5.1 Adapter registry

Adapters acquire and normalise. They do NOT analyze.

```
AdapterInterface ■■■ adapter_id : str // stable id, e.g. "url.acquire.v1" ■■■ accepts : Set[format] // e.g. {"html", "pdf", "docx"} ■■■ acquire(bytes|url|ref, ctx) -> AcquiredResource ■■■ health() : AdapterHealth
```

Initial registrations (map to existing code, no rewrite):

adapter_id	Current file	Role
------------	--------------	------

url.acquire.v1	services/ida/acquisition.py::acquire_url	Fetch URL
file.gridfs.v1	services/files/store.py::FileStore	Store + fetch uploaded bytes
sysmon.xml.v1	services/behavioral/sysmon_adapter.py::SysmonXMLAdapter	Sysmon XML → canonical
sysmon.evtx.v1	services/behavioral/evtx_reader.py::EvtxReader	EVTX bytes → Sysmon XML
archive.zip.v1	services/files/store.py::safe_iter_zip_members	ZIP members
pdf.text.v1	pdfplumber (inline)	PDF → text
docx.text.v1	inline zip-XML extraction	OOXML → text
image.acquire.v1	(new binding of existing services/adapters/image_adapter.py)	Image bytes → PIL image + metadata
text.passthrough.v1	(trivial)	pass-through for pasted text

No new adapter is written for this design. Existing code is re-labelled by registration.

5.2 Analyzer registry

Analyzers interpret. They do NOT acquire, do NOT decide routing.

```
AnalyzerInterface ■■■ analyzer_id : str // e.g. "die.command.v1" ■■■ accepts : Set[format] ■■■
analyze(input, ctx) -> AnalyzerResult ■■■ emits_provenance : bool
```

Initial registrations:

analyzer_id	Current file	Role
die.command.v1	services/die/api.py::analyze	Command / script AST + LOLBAS + IOC + technique
die.recursive.v1	services/die/recursive_decode.py	Recursive decoder
report_extractor.v1	services/ida/report_extractors.py	Arbitrary all commands/IOCs/MITRE/actors
image.ocr.v1	services/adapters/image_adapter.py	Image Adapter (read-code today, still not authorised)
csv.edr.symantec.v1	services/die/csv_edr_analyzer.py	Symantec SEP CSV
ioc_enrichment.v1	(composite of atomic-IOC + TI lookup)	Reputation enrichment only
pe.header.v1	services/pe_analyzer.py	PE header/section (currently orphan)
narrative.canonical.v1	services/die/canonical_narrative_enricher.py	Canonical narrative → MITRE
mitre.regex_diag.v1	operations.mitre_map	Regex mapper — DIAGNOSTIC ONLY (not authoritative)
verdict.risk_score.v1	operations.risk_score	Verdict scorer

None of these are rewritten during design. The registry captures what already exists and gives it a stable id.

5.3 Registry discipline

- Ids are immutable once registered. Version bumps produce new ids (e.g. `die.command.v2`).
 - The registry is the ONLY route from the IUE plan to executable code. Nothing dispatches by prose.
 - The registry is deterministic (Python module import time). No dynamic discovery.
 - Every registered entry declares its `accepts` set; the router cross-checks against `content.facets[].kind`.
-

§6 · URL content decomposition — design

This is the model that would eventually cover the SystemWeakness case.

6.1 Two-stage decomposition

```
Stage 1 – Acquisition (adapter) url.acquire.v1 → AcquiredResource AcquiredResource contains:
article_text, structured_blocks, html_dom, image_refs, outbound_links, media_types, http_status,
acquisition_strategy Stage 2 – Content Decomposition (IUE re-run) IUE re-runs against
AcquiredResource emits a REFINED InputUnderstanding whose content.facets = [ {kind: text,
observed_ref: block[0]}, {kind: encoded, observed_ref: block[5], extraction_hint: "base64 in
<pre>"}, {kind: image_ref, observed_ref: img[0], extraction_hint: "src=...screenshot.png"}, {kind:
image_ref, observed_ref: img[1]}, {kind: artifact_ref, observed_ref: outbound_link[3],
extraction_hint: "linked .zip"} ]
```

Design rule: the IUE runs TWICE for URL content — once on the URL itself (deciding to acquire), once on the acquisition result (deciding what facets to process). Both invocations emit an InputUnderstanding; the second one has `source.origin=reursion` and `source.parent_ref` pointing at the first.

6.2 Facet-level fan-out plan

For each facet the router dispatches:

```
text → analyzer_id: report_extractor.v1 encoded → analyzer_id: die.recursive.v1 →
die.command.v1 image_ref → adapter_id: image.acquire.v1 → analyzer_id: image.ocr.v1 → IUE
re-run on OCR text → recurse artifact_ref → adapter_id: url.acquire.v1 (if reachable) → IUE
re-run on bytes → recurse
```

6.3 Merge back to a single evidence set

All facet-level evidence records converge at the router's `final_convergence` sink. Each record carries `provenance.step_id` and `provenance.facet_kind` so the Workspace can present them grouped by origin OR flat.

6.4 What this design does NOT do

- Does NOT authorise OCR to be wired in. That's Task M8-adjacent, not this design.
 - Does NOT enable the Playwright fallback. That's an environment change.
 - Does NOT extend `_VENDORS`. Vendor-recognition becomes a *classifier input* to `source.vendor` and does not gate acquisition — instead, IUE emits `intent.acquire=true` for any input where `content.has_text=true` or `content.has_images=true` (i.e. any content-carrying URL). Then M2/M3 (UA + Playwright) become the transport tuning problem, no longer a policy problem.
-

§7 · Recursive artifact re-entry — design

7.1 Loop shape (design)

```
raw_input → IUE(1) (source.origin = pasted/uploaded/telemetry) → Router executes plan → analyzer emits evidence AND CHILD_ARTIFACTS (e.g. decoded base64 layer, extracted ZIP member, referenced instructions.docx, decoded XOR blob) → router loops each CHILD_ARTIFACT back through IUE(N+1) with source.origin = "recursion", source.parent_ref = step_id → IUE(N+1) emits fresh plan for the child → Router executes → ... → convergence at fixed point (see 7.2)
```

7.2 Termination conditions

- Maximum recursion depth: 5 (matches current recursive_decode.py:158 cap; retained as a **contract** rather than an implementation detail).
- Content-hash de-duplication: a child artifact whose envelope.evidence_ref matches an ancestor's is dropped.
- Any child that yields IUE intent.inspect_only=true (i.e. an atomic IOC) terminates cleanly at IOC enrichment.
- Any child that fails to acquire terminates at a canonical evidence record marked `provenance.decisions[].evidence = "acquisition_failed"` — **the failure is captured, not silent.**

7.3 Where "the DOCX referenced by the Python -c XOR loop" would go

Under the design:

```
Input python -c "exec(base64.b64decode(...).decode())" IUE(1) type=text format=python
content.facets=[{kind:text},{kind:encoded}] intent.decode=true intent.decompose=true Router
analyzer_id=die.command.v1 + die.recursive.v1 die.recursive.v1 peels one base64 layer → decoded
Python Analyzer Python analyzer emits CHILD_ARTIFACTS = [ {kind: artifact_ref, name:
"instructions.docx"} ] Router loops CHILD back to IUE(2) IUE(2) type=file format=docx
source.origin=recursion source.parent_ref=step_1 content.facets=[{kind:artifact_ref,
observed_ref:"instructions.docx"}] intent.acquire=false (bytes not locally available)
intent.inspect_only=true (record the reference; can't open the file) Router terminate at IOC
enrichment with an "unresolved child artifact" record.
```

The user is told: *"the payload references instructions.docx, which is not bundled with the sample. Reference recorded; content not analysed."* The failure is **explicit** and **explanatory**.

§8 · Provenance model — canonical evidence extension (DESIGN ONLY)

8.1 Extension shape

Every canonical evidence record gains a small provenance block:

```
canonical_evidence_record ■■■ ... existing fields (source, event_or_rule, field, observed_value,
evidence_ref, confidence, raw_refs) ... ■■■ provenance ■■■ step_id : str // the router step that
produced it ■■■ adapter_id : Optional[str] ■■■ analyzer_id : Optional[str] ■■■
extraction_method : enum(html_body | image_ocr | archive_member | ■ decoder_layer |
telemetry_field | ■ ast_match | regex_match | recursion) ■■■ parent_ref : Optional[str] //
parent artifact evidence_ref ■■■ location : Optional[str] // e.g. "url#img[2]",
"docx:word/document.xml" ■■■ source_confidence : float // trust in the extraction source ■■■
extraction_confidence: float // trust in the extraction itself
```

8.2 Discipline

- Every record MUST carry provenance. Records without it are rejected by the sink.
- OCR-derived records MUST have `extraction_method="image_ocr"` and cannot be merged with body-text records that share the same `observed_value` **unless** provenance is preserved on both sides (dual-witness rule).
- Two records that share the same `observed_value` but differ only in `location` are de-duplicated with `count += 1` and `raw_refs.append`, exactly as `sysmon_adapter` already does for Event 3.
- OCR text can never silently replace authoritative body text. Both are stored; the Workspace can filter.

8.3 Backward compatibility

Existing evidence records (Sysmon E1/E3, DIE, CSV/EDR) that don't yet carry provenance are treated as `provenance = None`. The sink accepts them but stamps `provenance.extraction_method = "legacy_unknown"`. Migration is opt-in per producer.

§9 · Failure-state model — design

The audit found six failure modes collapsing into one visible outcome. The target IUE + Router surface them explicitly.

9.1 State model

Every step in the plan can end in one of these states:

```
StepOutcome ∈ { ok, ok_partial, // some evidence, some failures
  acquire_failed, // adapter
  couldn't get bytes
  acquire_blocked, // 403/429/robots
  acquire_unavailable, // network / DNS /
  Playwright
  parse_failed, // adapter got bytes but couldn't normalise
  analyze_failed, // analyzer
  error
  analyze_skipped, // IUE said not to analyze
  empty, // analyzer ran but found nothing
  dependency_missing, // prerequisite step failed and failure_policy=required }
```

The Workspace projection can then say:

- "URL acquired, article extracted, no commands found" → `ok_partial + analyze_empty`
- "URL fetch returned 403; fallback engines unavailable" → `acquire_blocked + acquire_unavailable`
- "URL classified as atomic IOC by policy — no acquisition attempted" → `analyze_skipped` with `intent.inspect_only=true` in provenance

No more silent empty investigations. Every empty case ends in a `StepOutcome` with a human-readable reason.

9.2 Where the outcomes surface

- Router adds `step_results[step_id] = StepResult(outcome, reason, artifacts, evidence)`.
- Case save persists `case.step_results` alongside the existing `case.output`.
- Workspace shows a Failure & Fallback chip in every panel that would otherwise be empty.

§10 · Workspace convergence model — design

10.1 Target: one canonical response

Today the Workspace merges 5 independent responses (`/decode/smart`, `/die/understand`, `/die/analyze`, `/die/narrate`, `/die/investigation-results`). Target: **one endpoint that returns one canonical bundle**.

```
POST /api/investigate (target - NOT implemented) Body: { input?: str, case_id?: str, upload_ref?: str } Response: { understanding: InputUnderstanding, // $2 plan: List[PlanStep], // $2.7 step_results: Dict[step_id, StepResult], // $9 evidence: List[CanonicalEvidenceRecord], // $8 mitre: AuthoritativeMitre, // UI-DEF-02 authoritative only verdict: Verdict, // single scorer narrative: AnalystNarrative, // canonical enrichment provenance: ProvenanceBundle, // $8 }
```

10.2 Rules

- Workspace does NO client-side merging. It renders-projects the single bundle.
- Panels that don't apply (e.g. Behavioral Timeline for a URL input) simply don't render — no wasted round trips.
- Behavioral evidence remains a separate lane (`/api/behavioral/*` retains its identity) — but its output plugs into the same `evidence[]` array of the canonical bundle when a case id ties them together.

10.3 Legacy `/api/decode/smart` and siblings

- They are NOT deleted. They are marked deprecated=true with a 60-day sunset window (per ADR-0009 §5.1 pattern).
- During deprecation they proxy through the new `/api/investigate` endpoint and return a shape-compatible response for existing clients.

§11 · Migration dependency graph

[illegible]

Key ordering rule: no acquisition, UA, Playwright, or OCR change until the router + registry + provenance blocks are done, because those are the layers that make each subsequent change *safe*.

§12 · M1–M8 implementation sequence (target order)

#	Migration	Depends on	Risk	Effort	Owner-gate
D0	Design (this document)	ADR-0013 audit	none	—	■ NOW
M0a	Freeze current IUE contract as stable projection	D0	LOW	1 pass	■ awaits owner

M0b	Register existing adapters & analyzers by id (no code changes)	D0	LOW	1 pass	■
M0c	Extend canonical evidence with additive provenance block (nullable)	D0	LOW	1 pass	■
M0d	Ship thin Router (registry-driven dispatcher; no new analyzer)	M0a + M0b	MED	2 sessions	■
M0e	Ship IUE-v3 dataclass as additive fields on the current one	M0a	MED	2 sessions	■
M0f	Fold <code>nivxforge/investigation/input_understanding.py</code> and <code>canonical/iue/*</code> into <code>services/die IUE</code>	M0e	MED	1 session	■
M0g	Introduce <code>/api/investigate</code> (canonical single-response) that internally calls Router	M0d + M0e	MED	2 sessions	■
M0h	Workspace client switches primary Investigate button to <code>/api/investigate</code> ; deprecate <code>/decode/smart</code> (proxy shim)	M0g	HIGH	1 session	■
M1	Vendor catalogue merged into IUE <code>source.vendor</code> classifier input (<code>_VENDORS</code> becomes data, not gate)	M0e	LOW (data)	1 file	■
M2	Adapter <code>url.acquire.v1</code> gets browser User-Agent + fall-forward on 403/429	M0b	LOW	1 file	■

M3	Env: install Playwright chromium (preview + prod parity)	M2	LOW	env command	■
M4	<code>_next_engine("url")</code> retired — replaced by IUE <code>intent.acquire=true</code> for content-carrying URLs	M0b + M1	MED	1 file	■
M5	<code>_atomic_ioc_kind</code> retired — becomes an IUE input signal via <code>intent.inspect_only=true</code>	M0h	MED	1 file	■
M6	Legacy <code>operations.mitre_map</code> regex demoted from <code>verdict_card</code> to diagnostic-only (<code>mitre_provenance.regex_extra</code>) on all paths	M0h	MED	verdict_card change	■
M7	8-classifier consolidation cleanup (removal of dead-ish IUE #2, UIL classifier alias, <code>_atomic_ioc_kind</code> module, etc.)	M0h + M6	HIGH	multi-session	■
M8	Image acquire + OCR wire-in via <code>image.acquire.v1</code> + <code>image.ocr.v1</code> ; dual-witness provenance enforced	M0c + M0d	MED	2 sessions	■

All items above M0a–M8 are **LOCKED**. Only the design document (D0) is authorised.

§13 · Risk analysis

#	Risk	Layer	Mitigation
R1	IUE-v3 dataclass grows into a monolith	Contract	Hierarchical grouping (§2); only 6 top-level sub-nodes; per-node schema is small

R2	Router becomes another classifier (mission creep)	Router	Contract discipline: no <code>if content.kind == ...</code> branching in router; all decisions live in IUE
R3	Registry gets stale entries	Registry	<code>AdapterInterface.health()</code> + startup-time registration; CI test that every id resolves
R4	Provenance block adds cost on every record	Evidence	Provenance is small (10 scalars); records already carry <code>evidence_ref</code> and <code>raw_refs</code> ; delta is negligible
R5	Legacy /decode/smart shape drifts	Compat	M0h ships a shape-compat proxy shim; regression corpus locks the response bytes
R6	Workspace UI relies on prose engine labels	UI	Keep prose labels as <code>plan.steps[].reason</code> while adding machine ids elsewhere
R7	Determinism regresses when IUE runs twice for URL content	Determinism	Both invocations are cache-keyed on <code>envelope.evidence_ref</code> ; same input $\Rightarrow$ same bundle
R8	Deprecation friction (existing saved cases with old shape)	Persistence	Case-loader reads either shape; new fields default to None when absent
R9	Failure-state chips leak private URLs to shared UI	Privacy	Redact <code>source.locator</code> in projections shown outside the owning case's tenant
R10	5-level recursion cap becomes semantic contract, breaks a real corpus that needs 6	Recursion	Design allows the cap to be raised via <code>ExecutionContext.max_recursion_depth</code> ; the default (5) is a contract constant, not a hard limit
R11	The audit found 8 competing classifiers; someone adds a 9th before M0f	Governance	Add a lint rule in CI: any new <code>classify*</code> function must import from <code>services/die/input_understanding</code> or fail the build
R12	Shadow-only adapters (<code>services/adapters/*</code>) atrophy further	Registry	M0b registers them; CI test asserts each registered adapter is import-clean
R13	AI describe leg still adds MITRE outside authoritative surface	MITRE integrity	Out of scope for this design; documented as a follow-up authority merge
R14	Behavioural evidence lane silently diverges from <code>/api/investigate</code>	Convergence	M0g bundles behavioural evidence into the same <code>evidence[]</code> when <code>case_id</code> ties them

R15	Analysts trained on today's Attack Chain semantics see subtle projection changes	UX	Behaviour of individual panels does NOT change during M0*; only the wire route changes. UI-DEF-02 authoritative MITRE surface remains fixed
-----	--	----	---

§14 · Backward-compatibility strategy

14.1 IUE consumers today

- `frontend/src/pages/WorkspacePage.jsx` — reads `input_type`, `hero`, `confidence`, `plan[].description`, `execution_trace[]`, `engines_selected[]`, `engines_skipped[]`, `pipeline_flow[]`.
- `backend/services/die/investigation_results.py` — reads `input_type`.
- `backend/services/die/canonical.py` — reads `plan[]`, `input_type`.
- `backend/canonical/iue/adapters/text_structure.py` — reads `input_type` via a bridge.
- `backend/routers/ops.py:2495` — stamps the IUE result onto `cio.metadata.input_understanding`.

14.2 Rule: additive, not breaking

- IUE-v3 dataclass adds `envelope`, `input`, `source`, `content`, `intent`, `plan_v3`, `provenance` as new top-level fields.
- Existing fields (`input_type`, `hero`, `confidence`, `reasoning`, `summary`, `decode_required`, `decode_next`, `decode_layers`, `plan`, `execution_trace`, `overall_status`, `engines_selected`, `engines_skipped`, `pipeline_flow`) remain, populated by an **automatic projection** from the new structure:
- `input_type` = derived from `input.format` for the current 21 values; other values become `plain_text` or `unknown`.
- `plan[].engine` = derived from `plan_v3.steps[].analyzer_id` via a fixed prose-label lookup table.
- `engines_selected/skipped/pipeline_flow` = derived from `plan_v3.steps[]`.

14.3 Case-load compatibility

- Existing saved cases carry the old `InputUnderstanding` shape only. The loader treats them as `{envelope: None, input: {type: infer, format: infer}, source: None, content: None, intent: None, provenance: None}` with the legacy fields intact.
- New saved cases carry BOTH shapes so downgrade is safe.

14.4 API contract compatibility

- `/api/die/understand` continues to return the current shape. It internally invokes the IUE-v3 code path but re-projects to the old shape.
- New endpoint `/api/investigate` returns the canonical bundle (§10). No client is forced to switch on day one.

§15 · Test strategy — design

15.1 Regression locks (already in place)

- 67/67 P2 + UI-DEF-02 pass (Task-2 baseline). Every design implementation session MUST re-prove this.

- Frozen 12-case regression corpus (ADR-0010n). MUST re-prove.
- Sample-1 immutability guard. MUST re-prove.
- Report determinism guard. MUST re-prove.

15.2 New test suites (proposed, not yet added)

Suite	Purpose	Where
<code>test_iue_v3_contract.py</code>	Locks the additive fields, deterministic hashing, projection back-compat, total-on-any-input	<code>backend/tests/canonical/iue/</code>
<code>test_router_thin_dispatcher.py</code>	Ensures router never decides — only dispatches; every registered id resolves	same
<code>test_registry_hygiene.py</code>	Every adapter/analyzer id present in registry has a live import; no orphans	same
<code>test_provenance_dual_witness.py</code>	OCR text and body text with same <code>observed_value</code> remain as TWO records, not one	<code>backend/tests/canonical/evidence/</code>
<code>test_recursion_fixed_point.py</code>	Recursion terminates within 5 levels; content-hash dedup fires; <code>parent_ref</code> preserved	<code>backend/tests/canonical/router/</code>
<code>test_failure_state_semantics.py</code>	Each of the 10 <code>StepOutcomes</code> surfaces with a human-readable reason	same
<code>test_two_iue_merge_equivalence.py</code>	The old <code>nivxforge</code> IUE and <code>services/die</code> IUE produce identical projections on the frozen corpus	same
<code>test_workspace_investigate_bundle</code>	<code>/api/investigate</code> shape lock (fields, ordering, size)	<code>backend/tests/canonical/api/</code>
<code>test_ratchet_no_new_classifier.py</code>	CI lint: no new <code>classify*</code> function outside <code>services/die</code> IUE	<code>tools/</code>

15.3 Corpus additions (evidence-locked, deferred)

- **PrevMode** and **Examine** cases become permanent regression rows. Both currently fail (`Examine`'s empty, `PrevMode`'s under-called). They stay red until the migration lands, then flip green with owner-authorised expected behaviour.
- **Two more SystemWeakness-class URLs** (Medium / infosecwriteups) added to the corpus so the `vendor-catalogue-not-gate` design change is regression-locked.

15.4 Determinism ratchet

- Every migration step MUST reproduce a byte-identical response for every case in the frozen corpus (or explicitly opt out with an ADR-owned exception).
 - Response envelopes are compared via a canonicalised JSON hash to catch subtle key-ordering or float-formatting drift.
-

§16 · Non-goals (explicit)

To avoid drift, this design deliberately does NOT:

- Rewrite any adapter.
- Rewrite any analyzer.
- Add MITRE mappings.
- Change verdict-scoring formulae.
- Promote Verdict Engine v3.
- Expand IKG.
- Ship OCR.
- Ship image acquisition.
- Extend `_VENDORS`.
- Install Playwright.
- Change the User-Agent.
- Change the atomic-IOC guard's behaviour (only its *placement*).
- Touch the Behavioral Timeline persistence work from ADR-0010v.
- Reopen Sysmon Event 22 or Event 11.

§17 · Rules confirmation

- ■ No code, UI, IUE, router, adapter, canonical-evidence, or Workspace change performed.
- ■ No implementation.
- ■ No `_VENDORS`, User-Agent, Playwright, OCR, ImageAdapter, MITRE, verdict, or IKG change.
- ■ Task 2 (Real EVTX Fixture) remains COMPLETE and green.
- ■ Task 3 (auto-scroll) and Task 4 (source-agnostic audit) remain LOCKED.
- ■ ADR-0013 (IUE audit) remains the reference architecture-state anchor.
- ■ M0a–M8 are LOCKED. Only D0 (this design document) is authorised.

End of design. STOP. Awaiting explicit owner authorisation before any migration step.